Lab 06—Si, Señor y Señorita! Si! Si!

ReadyAVR C Programming

Overview, Hardware and Software Architectures

This lab builds on what you already know about interrupts, but we're leaving assembly language behind and moving on to C. And while C is quite different from AVR assembly, by using the predefined `#include` files in Atmel Studio you'll be able to access the internal hardware features of the ATmega128A in a very intuitive and familiar way. The goal of this lab is to create several "flashing patterns" using eight LEDs attached to the ReadyAVR board, and to then use the on-board joystick's center pushbutton to switch between those patterns.

Preliminary Hardware Setup

Your ReadyAVR should already have eight LEDs with pull-up resistors and connector pins soldered onto the board's prototyping area (see Fig. 1 below). If not, use this section to prepare the board.

If the LEDs are already installed, you will move to the Software Architecture section, but you **MUST answer** all questions from this section in your final report.

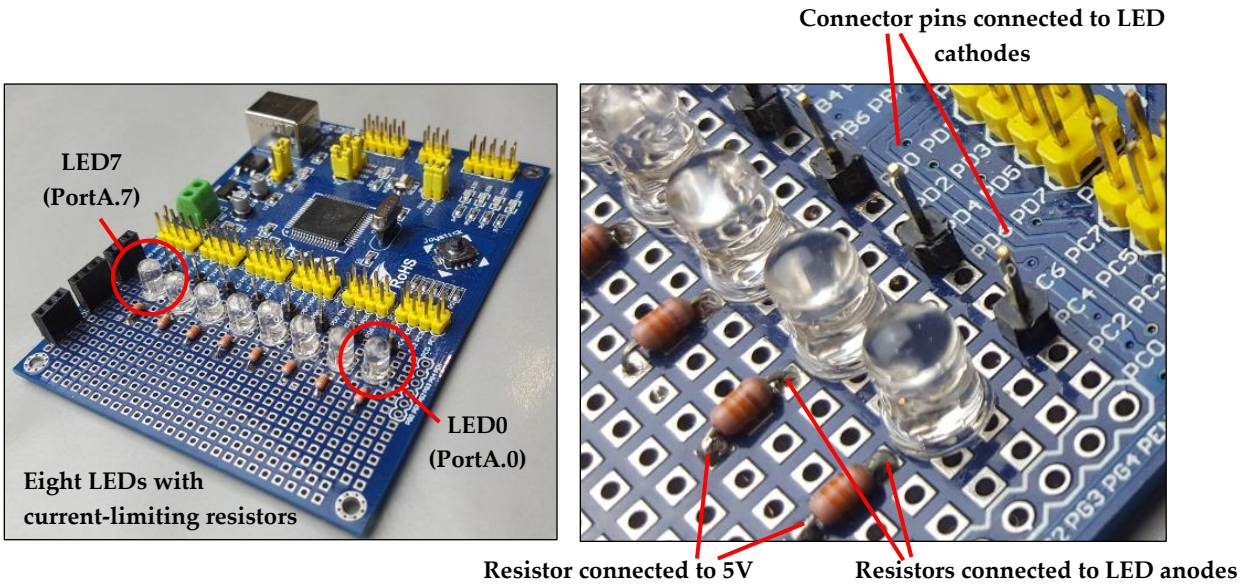


Figure 1 ReadyAVR with active-low LEDs and 220Ω current-limiting resistors.

You'll be using the ATmega128A to drive eight LEDs. When powering an LED, you must provide enough current to turn it on but not so much that you damage the LED or the ATmega128A. You do this by adding a *current-limiting resistor* in series with the LED. For the purposes of this lab **assume a safe LED operating current of 2.0mA to 20.0mA**.

Begin by opening the **ATmega128A Datasheet** (Canvas Files>Docs) and determine the max current available from the processor's I/O pins (Hint: this can be found in the "Electrical Characteristics" section of the datasheet).

Question 1: What is the max current available from a single ATmega128A I/O pin?

Next, use this max current rating along with a V_{CC} supply voltage of 5.0V and an LED voltage drop of 2.0V to **prove that it is safe to operate each LED using a 220Ω current limiting resistor**. Draw a **simple schematic of a single LED circuit** and show it to your instructor. Also, **include the design and calculations in your final lab report**.

Question 2: What is the simple LED circuit and is it safe?

Board Soldering (should not be required)

Once you've proved the circuit is safe, solder eight LEDs, current limiting resistors, and connector pins to the ReadyAVR board in a manner similar to that shown in Figure 1. **Ensure proper LED polarity.** Figure 2 shows the backside of the board with how the 5V power source, 220Ω current-limiting resistors, LEDs, and connector pins are attached.

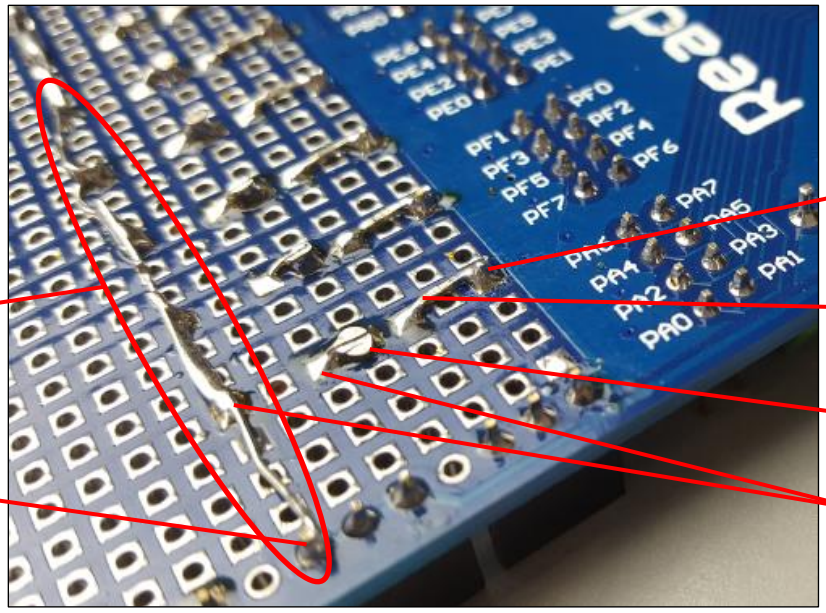


Figure 2 The backside of the ReadyAVR board showing connections

If you had to add resistors to your board, show your work to your instructor before continuing.

Software Architecture

You'll be using the ReadyAVR **joystick** again, but instead of controlling the blink rate of an LED, you'll use the joystick's **center switch** to change between four predefined flashing patterns for the eight LEDs externally connected to **PORTA** of the ATmega128A.

The patterns are:

1. Low to high (A0 to A7, with only one LED on at a time).
2. High to low (A7 to A0, with only one LED on at a time).
3. Back and forth (A0 to A7 to A0, with only one LED on at a time). Do **not** let the LEDs on each end stay on for double the amount of time!
4. Your choice. Make it interesting! Your instructor will show you some examples. Do more than one for some extra credit—see the Report section below for more info.

Important Code Design Information

The code to flash the LEDs must be in your `main()` routine, and an ISR, triggered by pressing the joystick button, should change a **state variable** that `main()` uses to decide which pattern to display. This design pattern is called the *reactor pattern* and should be familiar to you from your programming classes. A second ISR must be used to keep track of time in 1mS intervals or "Ticks." Use this "Tick" counter to **ensure that LEDs stay on for 50mS**. Study Figure 3 for more details on how this should be implemented.

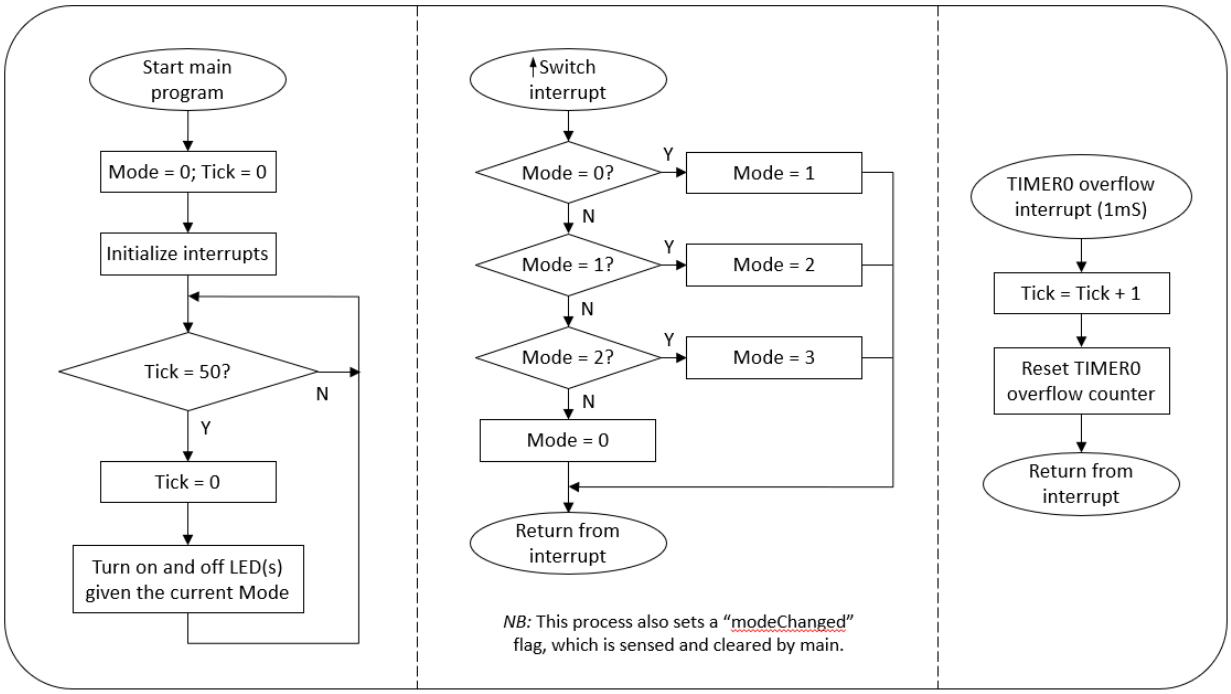


Figure 3 StateChart representation of the program's logic showing three concurrent processes.

Lab Procedure

Start by opening GitHub Classroom link and clone this repo to your local machine. Open the

In Microchip Studio, open up the `Lab06.cproj` and you will have a `Lab06.c` file that is your starter file. Your professor will walk you through the file at the start of lab. Note that the `#include` files are required to have this code work.

> *Professor Note: Your text book is NOT specific to the ATMega128A, so the example code has fewer interrupts and slightly different configurations than you need for your part. Cross check items with the datasheet.*

Using eight female-to-female jumpers, connect the eight LEDs on the ReadyAVR board to the eight pins on PORTA. **Be sure PortA.0 is connected to LED0** on the ReadyAVR board (check Figure 1 again), and **PortA.7 is connected to LED07**.

Your code development should follow steps similar to these:

1. Create an ISR to set a global state variable named "`Mode`" to 0, 1, 2, or 3 as you continue to press and release the center joystick button. **Only change** `Mode` **when the button is released**, and after 3 go back to 0. (The lab slides recommend using enum data structure… Review your C-textbook, or the one in the lab, if you need some review).

Professor Note: The center joystick button is wired to a pin that cannot generate an interrupt itself, so externally connect the `PORTB0` *pin associated with the center joystick switch to the* `PORTD0` *pin and use interrupt* `INT0` *to respond to release of the joystick's center pushbutton. (If this does not make sense, review Lab05!)*

2. Create a second ISR that fires **once per millisecond**. Use the **TIMER0 overflow** condition to trigger the interrupt, and inside this second ISR increment a global variable named `"Tick"`.
   Microprocessor Geek Note: Your TIMER0 interrupt handler will need to reload the TCNT0 register with its initial count value so that TIMER0 will overflow again in exactly 1mS. ***Very important****!*
3. Once your interrupt routines are working, write your `main()` program to implement the LED flashing patterns mentioned earlier. Your `main()` code should configure the interrupt registers and I/O lines, then enter an infinite loop (often called a *"game loop"*) that uses Mode and Tick to decide which pattern to use and which LED(s) to turn on and off every 50mS.
4. Figure 3 shows this logic. The "little" loop in `main()` spins until "Tick" reaches 50. Once 50 is reached, `main()` resets Tick to zero, determines which pattern (via "Mode") is being used, and then turns off the currently lit LED and turns on the next LED in the pattern.
   The larger loop in `main()` is the game loop! If this isn't 100% clear, review Figure 3. Still not clear? Ask!

Report

This is a **one-week lab**. For full points, you must

- **demonstrate your working code** to your instructor,
- **submit your lab report via Canvas,**
- **push your final code via GitHub**.

In your report, include a concise description of how TIMER0 is configured and used. In particular, show your calculations to confirm that the PRE-SCALER and TMRCNT used generate the correct 1ms interrupt.

As always, you'll be graded on the correctness, completeness, readability and organization of both your report and your code, so make it **read well** and **look good**! Your code should also be properly indented and reasonably commented.

And if you do more than one of your own patterns, **you can earn bonus points**! How many points will depend on how many extra patterns you develop, how interesting

they are, how well they function, and how well you describe the design of each pattern in your lab report.

Have fun and blink on!